Ngadu Functional Specification

Status: Draft for review

Date: 2026-10-13

1. Purpose

Build Ngadu, a web app that helps a community of friends organize badminton sessions and create match pairings. Members sign in with Google and use phone or desktop browsers. Success means an organizer can arrange a session, identify players, and publish pairings that members can view.

2. Confirmed architecture

Area	Choice	Responsibility
Repository	Monorepo	Frontend, backend, and documentation in one Git repository
Frontend	Next.js, React, TypeScript	Pages, navigation, rendering, browser interaction
Frontend tooling and runtime	Bun	Dependencies and running the Next.js server
Backend language	Go	Separate API service and authoritative application rules
Authentication	Firebase Authentication	Google login and user identity
Application database	PostgreSQL	Profiles, memberships, sessions, matches
Schema migrations	Goose as a Go library	Versioned SQL migrations under backend/migrations
Development file storage	SeaweedFS	Uploaded files when uploads are included
Local infrastructure	Docker and Docker Compose	Local PostgreSQL; Next.js and Go run on the host for development

The backend will be a separate Go service. Its HTTP framework, API database library, and production build/deployment configuration remain undecided. Production hosting and storage remain undecided.

Firebase manages login identities. PostgreSQL stores application data linked to the verified Firebase UID. Firebase Authentication does not require Firestore. The Go backend must securely verify Firebase ID tokens.

3. Monorepo organization and current state

Directory	Purpose	Current state
frontend	Frontend with its own package manifest and Bun lockfile	Generated Next.js starter
backend	Separate Go backend	Health/readiness server and development seeder; application endpoints pending
docs	Functional specification and PDF export	Markdown is the source of truth

Root Bun scripts delegate to the frontend; the Makefile also provides Go and local database commands. Each service keeps its own toolchain. Sharing one repository does not require a JavaScript workspace or an extra build orchestrator.

The Go backend provides operational health/readiness endpoints. Goose SQL migrations define the current minimal schema. A separate development seeder inserts fixture data. Migration and seed commands are Go programs under backend/scripts; no standalone Goose installation is required. Docker Compose runs local PostgreSQL. Firebase integration and application API endpoints remain unimplemented; the frontend uses demo data. The following sections describe proposed application behavior.

4. Proposed first release

The following scope needs confirmation:

- One community initially, with members and organizers.
- Community access through invitations.
- Player profiles with display names and optional skill levels.
- Sessions with date, time, venue, and participants.
- Members joining or leaving open sessions.
- Organizers generating, reviewing, and publishing match pairings.
- Members viewing published pairings.
- Manual refresh initially unless automatic updates are required.

Payments, receipt uploads, chat, tournaments, rankings, notifications, and automatic email invitations are deferred. Score recording and custom profile uploads are optional extensions. SeaweedFS is selected for development but does not itself require an upload feature in the first release.

5. Users and permissions

User state	Proposed permissions
Visitor	Sign in and view a valid invitation's limited preview
Signed-in nonmember	Accept a valid invitation or switch accounts
Member	View community sessions and pairings, edit their profile, join or leave open sessions
Organizer	Member actions plus managing invitations, sessions, participants, and pairings

Google sign-in establishes identity; community membership grants access. Every protected backend request checks identity, membership, and any required role. Hiding a button is not an authorization check.

The first organizer is assigned through controlled setup using a known Firebase UID. Users cannot grant themselves organizer privileges. UI workflows for role management and member removal need a later decision.

6. Authentication and profiles

Google sign-in

1. The user chooses "Continue with Google."
2. The frontend completes sign-in through Firebase Authentication.
3. Protected API requests carry a Firebase ID token.
4. The backend verifies signature, issuer, audience, and expiry and identifies the user by the verified UID.
5. Members enter their community; nonmembers see an invitation-required state.

Cancelling login does not create membership. Failures show a useful message and allow retry. The frontend may refresh an expired token through Firebase; if authentication cannot be restored, the user signs in again. Signing out clears protected data from the current view.

A client-supplied user ID or email is not proof of identity. The Firebase UID, rather than email, links the identity to PostgreSQL.

Player profiles

Profiles contain an application ID, unique Firebase UID, display name, and optional photo reference. Skill level is proposed; its scale and effect on matchmaking remain undecided.

The app can use Google's photo URL without uploading a copy. PostgreSQL stores the reference. Unavailable photos fall back to initials or a default avatar. Custom pictures need the optional upload feature.

7. Invitations and membership

Invite-only access remains a proposed policy. If selected, the initial approach uses shared links that organizers can disable.

Create and open an invitation

An organizer creates a URL such as `/invite/<opaque-token>` and shares it through group chat or email. Automatic email sending is unnecessary. Next.js opens a join page and asks the backend to validate the invitation.

Valid invitations show a limited community preview and sign-in or "Join community." Invalid, expired, disabled, or exhausted links show an unavailable message. Existing members can enter the dashboard without consuming another invitation use. The preview never exposes members or session details.

Tokens are unpredictable, stored as digests, and excluded from normal logs. Shared links can be forwarded. Optional expiration and usage limits need chosen defaults.

Accept an invitation

1. Open the invitation page.
2. Sign in with Google if necessary, preserving the pending invitation across login.
3. Explicitly choose "Join community."
4. The backend verifies the login token and revalidates the invitation.
5. Membership and invitation usage are saved atomically in PostgreSQL.
6. The app opens the dashboard and requests missing profile information if needed.

Membership is unique per user and community. Repeated acceptance does not duplicate membership or consume extra uses. Concurrent acceptance respects configured usage limits. Opening a link or signing in does not automatically join the community.

Personal links, approved email lists, and organizer approval are alternative policies requiring a separate choice.

8. Sessions and participation

An organizer creates a session with a title, start date and time, timezone, and venue. Optional capacity and court count need confirmation. Store the start instant and session timezone and display the timezone clearly.

Members can join or leave open sessions. Repeated requests do not duplicate participation, and concurrent joins cannot exceed configured capacity. Full sessions show an explanation; waitlists are deferred. Organizers can manage participation on members' behalf.

State	Proposed behavior
Open	Members join or leave; organizers prepare and publish pairings
Active	Roster locked; pairings visible
Completed	Session and pairing history read-only
Cancelled	Cancellation visible; joining and play blocked

Only organizers change state. Starting requires published pairings. Completing an active session or cancelling an open or active session preserves history. Reopening and corrections to active rosters need explicit policies.

9. Matchmaking

The proposed workflow separates generating drafts from publishing pairings. The algorithm is undecided.

Generate a draft

An organizer requests pairings for the current roster. The backend returns assignments and explicitly identifies resting or unassigned players. Each round must satisfy these constraints:

- Only eligible session participants appear.
- No player appears in multiple simultaneous matches.
- Singles needs two distinct players; doubles needs four.
- Simultaneous matches cannot exceed configured court availability.
- Too few players produce an explanation rather than incomplete teams.

Select singles or doubles, skill balancing, partner/opponent rotation, rest distribution, and tie-breaking before implementing generation. No fairness guarantee applies until those rules are defined.

Review and publish

Organizers review drafts before publication. Members see published teams, opponents, rounds, and courts when configured; drafts remain organizer-only.

Publication rechecks the roster and pairing constraints. Stale drafts require regeneration or revalidation. Regenerating a draft never silently changes published matches.

The proposed baseline publishes one round at a time. Later rounds, corrections to published assignments, match completion, and scores need explicit decisions before inclusion.

10. Pages and interaction

Page	Purpose
Sign in	Google login and authentication errors
Invitation	Limited preview, link status, sign-in, explicit joining
Access required	Nonmember state and account switching
Dashboard	Upcoming sessions and profile navigation
Profile	Display name, photo reference, optional skill level
Session detail	Time, venue, roster, joining/leaving, published pairings
Organizer session controls	Settings, participants, draft generation, publication
Organizer invitations	Create, copy, inspect, disable links

Support phone and desktop layouts, keyboard navigation, labeled inputs, and loading, empty, success, and error states. Prevent accidental repeated submissions and retain input after recoverable failures. Styling and reusable components remain unselected.

11. Data and service boundaries

Record	Main information
User	Application ID, unique Firebase UID, display name, optional email/photo reference
Community	ID and name
Membership	Community, user, role; unique community-user pair
Invitation	Community, token digest, creator, status, optional expiry/usage settings
Session	Community, details, start instant, timezone, state, optional capacity/courts
Participation	Session and user; unique session-user pair

Record	Main information
Pairing draft	Session, roster version, round, assignments
Published match	Session, round, teams/players, optional court, publication metadata
File asset if enabled	Object identifier, owner, media type, size

This is a conceptual model, not a final schema. Invitation acceptance, capacity checks, and publication require transaction-safe concurrency controls.

Next.js handles rendering and browser interaction. The separate backend owns authoritative application rules, token verification, permissions, PostgreSQL operations, and authorized uploads. Browsers do not access PostgreSQL directly.

REST with JSON is proposed, not selected. API schemas, session transport, and whether to use a same-origin proxy or a separate API origin remain open. Backend language is Go; the HTTP framework and remaining API tooling stay undecided.

12. Optional uploads and access protection

If uploads are selected, the backend authorizes them and restricts file type and size. SeaweedFS stores development files; PostgreSQL stores object references and ownership metadata, not image bytes.

Private files require authorized retrieval. Public delivery, size limits, formats, replacement cleanup, retention, and production storage must be chosen before enabling uploads.

All private community reads and writes require membership checks. Guessing an identifier cannot bypass authorization. Database, storage, and server authentication secrets remain server-side. Login and invitation tokens stay out of normal logs.

The API distinguishes unauthenticated requests, denied access, invalid input, missing resources, state conflicts, and unavailable services. UI errors offer practical next actions without exposing internals. Separate origins need explicit allowed-origin settings; session transport determines CSRF protections.

13. Development and deployment

`make dev` builds the Go backend, starts PostgreSQL through Docker Compose, waits for database health and backend readiness, and runs the backend and Next.js locally. Ctrl+C stops both app processes; PostgreSQL can be stopped separately with `make db-stop`. Its named volume persists across ordinary stops and restarts. `make seed` (also `make db-seed`) starts, migrates, and seeds the same local development database. `make db-migrate` applies migrations, `make db-status` shows migration status, and `make db-revision msg=NAME` creates a SQL template. All Make service/database commands target local development and ignore ambient DATABASE_URL. The Bun supervisor lives under backend/scripts. Root `.env` or shell variables configure local ports and development database credentials. Frontend/backend container images and SeaweedFS orchestration remain pending. Production storage and deployment remain undecided.

Development login may use a real Firebase development project, the Firebase Auth emulator, or both. Emulator tests simulate identities and do not prove genuine Google sign-in works. Production credentials and data stay separate from development.

Production hosting, domain, backups, restore procedures, and monitoring are undecided. Repository publication does not deploy the application.

14. Acceptance scenarios

These scenarios validate the proposed baseline once its scope is accepted:

1. Google sign-in identifies the user without automatically granting membership.
2. A valid invitation survives login and creates one membership after explicit acceptance.
3. Disabled or expired links fail even if their page was opened earlier.

4. Repeated/concurrent acceptance preserves membership uniqueness and invitation limits.
5. Nonmembers cannot fetch private sessions through direct API requests.
6. Joining/leaving preserves unique participation and configured capacity.
7. Members cannot perform organizer operations through the UI or API.
8. Pairings contain eligible players without simultaneous duplicates or incomplete teams.
9. Roster changes require draft revalidation or regeneration before publication.
10. Members see published pairings and cannot access unpublished drafts.
11. Completed/cancelled sessions reject prohibited mutations and retain history.
12. Missing photos show a fallback; recoverable errors permit retry.
13. Invalid/expired identity tokens fail, and valid tokens never bypass membership checks.

15. Decisions needed before implementation

Decision	Open question
Backend tooling	HTTP framework, API database library, production build/deployment configuration
Membership	Invite-only, organizer approval, or another policy
Invitations	Shared/personal links, expiration and usage defaults
Community scope	One or multiple communities
Organizer management	Setup only or a role-management UI
Participation	Self-service joining, capacity, roster locking
Match format	Singles, doubles, or both
Pairing priorities	Skill balance, variety, rest fairness, tie-breaking
Pairing lifecycle	Later rounds, corrections, match completion
Scores	Include now or defer
Updates	Manual refresh, polling, or immediate updates
Styling	CSS approach and component library
API contract	API style, schemas, routing, session transport
Development login	Real Firebase project, emulator, or both
Uploads	Whether the first release needs uploaded files

Confirmed technology choices remain stable while these questions are resolved. Proposed features and unselected backend tooling are not implied commitments.